PRD — Natural Science Formula Reading Helper (NSF-Reading-Helper)

> **자연과학 공식 읽기 도우미**: 수학·물리·화학·지구과학·생물 등 자연과학 전반의 수식·기호를 **렌더링**, **음성/문자 읽기**, **명칭/뜻 설명**, **분야별 의미 비교**까지 한 화면에서 제공하는 교육형 도구. 웹 버전과 데스크탑 앱(Tauri 2 기반, **As It Sounds** 구조 준용)을 동시에 제공한다.

---

0. 문서 메타

항목	값
문서 ID	PRD-NSFRH-001
버전	0.1 (초안)
작성일	2026-05-19
작성자	NSF-Reading-Helper 팀
상태	Draft — 1차 리뷰 대기
참조	`.doc/Perplexity-수학계열, 물리,화학,지구과학,생물 등의 자연과학 포함하여 다양한 수식.md`, `~/Developer/NGL(Go,Rust,...)/As-It-Sounds` (구조 레퍼런스)

---

1. 배경 및 문제 정의

1.1 문제

- 자연과학 학습자·연구자·교사·시각장애 보조 사용자는 수식을 **시각적으로만** 접하며, "이 기호가 무엇이고 어떻게 읽으며 어떤 뜻인지"를 즉시 알 수 없는 경우가 많다.
- 동일 기호(예: μ, σ, Σ)가 수학·물리·화학·생물·통계 등 **분야마다 다른 의미**를 가진다.
- 기존 도구는 렌더링(MathJax/KaTeX), 음성화(SRE), OCR(pix2tex)로 **분절**되어 있으며, 한국어 명칭·뜻·예문을 통합 제공하는 솔루션이 부재.
- 데스크탑 환경에서 **오프라인** 학습 도구가 필요한 사용자(교실, 인터넷 제약 환경, 시각장애 보조)가 존재한다.

1.2 기회

- MathJax(Apache-2.0) + Speech Rule Engine(Apache-2.0) + 자체 한국어 기호 사전 + 선택적 OCR(pix2tex, MIT)을 결합하면 **법적 위험이 낮은** 풀스택 구성이 가능.
- Tauri 2 + Rust 백엔드는 다음을 가능하게 한다:
  - 단일 React 코드베이스를 웹과 데스크탑 양쪽에 재사용 (참조: As-It-Sounds 구조)
  - 오프라인 SQLite 기반 기호 사전 임베드
  - macOS Universal / Windows / Linux 멀티플랫폼 배포

1.3 비전 한 줄

> "수식을 보면 즉시 **읽어주고**, 클릭하면 **설명해주는** 자연과학 학습 동반자."

---

2. 목표(Goals) / 비목표(Non-Goals)

2.1 Goals — V1.0

1. LaTeX, MathML, AsciiMath, Unicode 직접 입력 4가지 포맷의 수식을 **정확히 렌더링**.
2. 입력된 수식 전체와 부분 토큰의 **한국어 자연어 읽기** 생성 (예: `\int_0^1 x^2 dx` → "0부터 1까지 x 제곱을 x에 대하여 적분").
3. 수식 내 임의 토큰을 클릭하면 **명칭/한글 읽기/뜻/분야별 의미/예시**가 패널에 표시된다.
4. 수학·물리·화학·지구과학·생물·통계·논리 7대 도메인을 커버하는 **초기 시드 사전 500개 이상**.
5. 별칭·LaTeX 명령·Unicode·한국어·영어·오타까지 포괄하는 **퍼지 검색**.
6. **웹 버전**: 브라우저에서 동작, 회원가입/로그인 없이도 기본 기능 사용 가능.
7. **데스크탑 앱**: macOS / Windows / Linux 멀티플랫폼, As-It-Sounds 구조 준용, 완전 오프라인 동작.
8. TTS(Web Speech API / OS TTS) 연동으로 **읽기 음성 출력**.

2.2 Goals — V1.5 (스트레치)

1. 이미지에서 LaTeX 추출 (pix2tex/Pix2Text 연동).
2. 손그림 기호 검색 (Detexify형 UX).
3. 화학식 전용 표기(`\ce{...}` mhchem) 지원.

2.3 Non-Goals (V1)

- 수식의 **수치 계산**, 풀이, CAS(Symbolic computation). → 별도 제품군.
- 일반 한국어 음성 인식(STT). → As-It-Sounds 영역.
- 실시간 협업 편집(Google Docs류).
- 모바일 네이티브(iOS/Android) 앱. → V2 검토.
- 사용자 작성 콘텐츠의 공개 공유/SNS 기능.

---

3. 사용자 페르소나 & 핵심 시나리오

3.1 페르소나

ID	페르소나	동기	핵심 요구
P1	**고교/대학 초년생 학생**	자연과학 교과 학습	모르는 기호 즉시 의미 확인, 읽기 익히기
P2	**과학·수학 교사**	수업 자료 작성 및 시범	빠른 렌더링, 학생에게 읽기 시범
P3	**시각장애/난독 학습자 (접근성)**	수식 음성 청취	TTS 정확도, 키보드 네비게이션, ARIA
P4	**연구자/대학원생**	논문 PDF의 수식 해석	이미지 OCR 입력, 분야별 의미 분기
P5	**오프라인 환경 사용자** (교실, 군부대 교육)	인터넷 차단 환경	데스크탑 앱, 사전 임베드

3.2 핵심 시나리오

S1. "이 기호 뭐예요?" (P1) 1. 학생이 교과서에서 $\oint \mathbf{E} \cdot d\mathbf{A} = Q/\varepsilon_0$를 본다. 2. 웹앱에 `\oint \mathbf{E} \cdot d\mathbf{A} = \frac{Q}{\varepsilon_0}` 입력. 3. 렌더링 결과 위 `\oint` 클릭 → "닫힌 경로 적분 / 컨투어 인테그럴 / 가우스 법칙 등 폐곡면에서의 적분에 사용" 설명 표시. 4. "전체 읽기" 버튼 → "닫힌 곡면에서 E 벡터와 d A 벡터의 내적을 적분한 값은 Q를 입실론 영으로 나눈 것과 같다" 음성 재생.

S2. 수업 시범 (P2) 1. 교사가 데스크탑 앱을 프로젝터에 띄운다. 2. AsciiMath로 `sum_(i=1)^n i^2 = n(n+1)(2n+1)/6` 입력. 3. 분야별 패널에서 "통계의 시그마"와 "수학의 합 기호"를 비교 보여줌.

S3. PDF 캡처 (P4) 1. 논문 캡처 이미지를 드래그앤드롭. 2. OCR이 LaTeX 추출 → 자동 렌더링. 3. 토큰별 설명 + 분야 강제 필터(예: "물리만 보기").

S4. 접근성 (P3) 1. 키보드 Tab으로 수식 토큰을 순회. 2. 포커스된 토큰의 ARIA-label에 한국어 명칭+뜻이 노출. 3. Enter 키 → TTS 읽기.

S5. 오프라인 수업 (P5) 1. 데스크탑 앱 설치(인터넷 1회). 2. 이후 SQLite 임베드 사전으로 완전 오프라인. 3. 모델 업데이트는 "사전 팩(.nsfpack)" 파일로 수동 갱신.

---

4. 기능 명세

4.1 기능 목록 (Feature Map)

ID	기능	V1	V1.5	우선순위
F-IN-01	LaTeX 입력	✅		P0
F-IN-02	MathML 입력	✅		P1
F-IN-03	AsciiMath 입력	✅		P1
F-IN-04	Unicode 기호 직접 입력 (∑, ∫, √…)	✅		P0
F-IN-05	이미지 OCR (pix2tex)		✅	P2
F-IN-06	손그림 기호 검색 (Detexify형)		✅	P3
F-RD-01	MathJax 렌더링	✅		P0
F-RD-02	토큰 클릭 인터랙션	✅		P0
F-RD-03	토큰 호버 툴팁	✅		P1
F-RD-04	출력 SVG/PNG 다운로드	✅		P2
F-SP-01	SRE 기반 전체 읽기 (한국어)	✅		P0
F-SP-02	SRE 기반 전체 읽기 (영어)	✅		P1
F-SP-03	단계별/부분식 읽기	✅		P1
F-SP-04	TTS 음성 출력 (Web Speech / OS)	✅		P1
F-SP-05	읽기 속도/음성 선택	✅		P2
F-EX-01	토큰 설명 패널 (명칭/읽기/뜻)	✅		P0
F-EX-02	분야별 의미 분기 표시	✅		P0
F-EX-03	예문/예시 수식	✅		P1
F-EX-04	유사 기호 비교 (∈/∋/⊂/⊆)	✅		P1
F-SR-01	기호명/별칭/LaTeX/Unicode 통합 검색	✅		P0
F-SR-02	pg_trgm/SQLite FTS 퍼지 검색	✅		P1
F-SR-03	분야 필터, 난이도 필터	✅		P2
F-LB-01	즐겨찾기 / 학습 노트	✅		P2
F-LB-02	최근 분석 히스토리	✅		P1
F-LB-03	노트 Export (Markdown/PDF)		✅	P2
F-DK-01	데스크탑: 오프라인 SQLite 사전	✅		P0
F-DK-02	데스크탑: 사전 팩(.nsfpack) 가져오기/내보내기	✅		P1
F-DK-03	데스크탑: 시스템 트레이, 글로벌 단축키		✅	P2
F-DK-04	데스크탑: 클립보드 LaTeX 자동 감지		✅	P2
F-A11Y-01	ARIA, 키보드 네비, 고대비 모드	✅		P0

4.2 기능 상세

F-IN: 입력층

- **4가지 입력 포맷 자동 감지** — 첫 문자(\\, <math, , Unicode)와 휴리스틱으로 LaTeX/MathML/AsciiMath/Unicode 분기.
- **샘플 라이브러리** — 분야별 예시 수식 패널에서 클릭 한 번 삽입.
- **입력 정규화** — 모든 입력은 내부적으로 **MathML(canonical)**로 변환 후 토큰화.

F-RD: 렌더링층

- **MathJax v4** 사용. `output: chtml` 우선, 접근성 옵션(`a11y/sre`) ON.
- 렌더링된 DOM에 `data-token-id`, `data-symbol-id` 주입 → 클릭/호버 이벤트 핸들링.
- 다크/라이트 테마, 폰트 크기 조절(80%~200%).

F-SP: 음성/읽기층

- **Speech Rule Engine (SRE)** — Clearspeak / Mathspeak 규칙 선택.
- 한국어 음성 규칙은 **자체 규칙 팩 (`ko-clearspeak.json`)**으로 SRE 확장.
- TTS 백엔드:
  - 웹: `window.speechSynthesis` (Web Speech API).
  - 데스크탑: OS TTS (macOS `say`, Windows SAPI, Linux `espeak-ng`/`speech-dispatcher`) — Rust 측에서 호출.
- **하이라이트 동기화** — 읽는 토큰을 시각적으로 강조 (As-It-Sounds의 세그먼트 하이라이트 패턴 차용).

F-EX: 설명층

- 토큰 클릭 시 우측 패널에 표시:
  - **명칭** (한/영)
  - **한글 읽기** ("인테그럴", "시그마")
  - **간단 정의**
  - **분야별 의미** (수학/물리/화학/…)
  - **예시 수식 3개** (난이도별)
  - **유사 기호** (비교 가능한 3~5개)
- 데이터 소스: 자체 구축 `symbols` + `symbol_aliases` + `symbol_domain_map` + `symbol_examples` 테이블.

F-SR: 검색층

- 검색창 한 줄로 다음을 동시 매칭:
  - 한글 명칭("적분", "시그마")
  - 영어 명칭("integral", "sigma")
  - LaTeX 명령(`\int`, `\Sigma`)
  - Unicode(∫, Σ)
  - 별칭/동의어/오타("인테그럴", "시그머", "")
- 백엔드:
  - 웹: PostgreSQL + `pg_trgm` trigram 인덱스 + Full Text Search.
  - 데스크탑: SQLite FTS5 + Levenshtein 보정 모듈.

F-LB: 학습 도구

- **즐겨찾기** — 사용자 계정 기준(웹) 또는 로컬 DB(데스크탑).
- **노트** — 수식 + 직접 작성 메모 + 분야 태그.
- **히스토리** — 최근 100개 분석 결과 자동 저장.

F-DK: 데스크탑 전용

- **오프라인 사전 임베드** — 설치 시 기본 1,000+ 기호 시드 SQLite로 동봉.
- **`.nsfpack`** — 사전 패키지 포맷(zip + manifest.json). 사용자가 직접 사전을 가져오거나 내보낼 수 있음.
- **클립보드 자동 감지** — LaTeX 패턴 클립보드 감지 시 트레이 알림.
- **글로벌 단축키** — `Cmd/Ctrl+Shift+M`으로 빠른 입력창.

F-A11Y: 접근성

- WCAG 2.1 AA 기준.
- 모든 인터랙티브 요소에 ARIA.
- 수식 토큰은 `role="math"`, `aria-label`에 한국어 읽기.
- 키보드 단축키 지원, 포커스 visible.
- 고대비 테마.

---

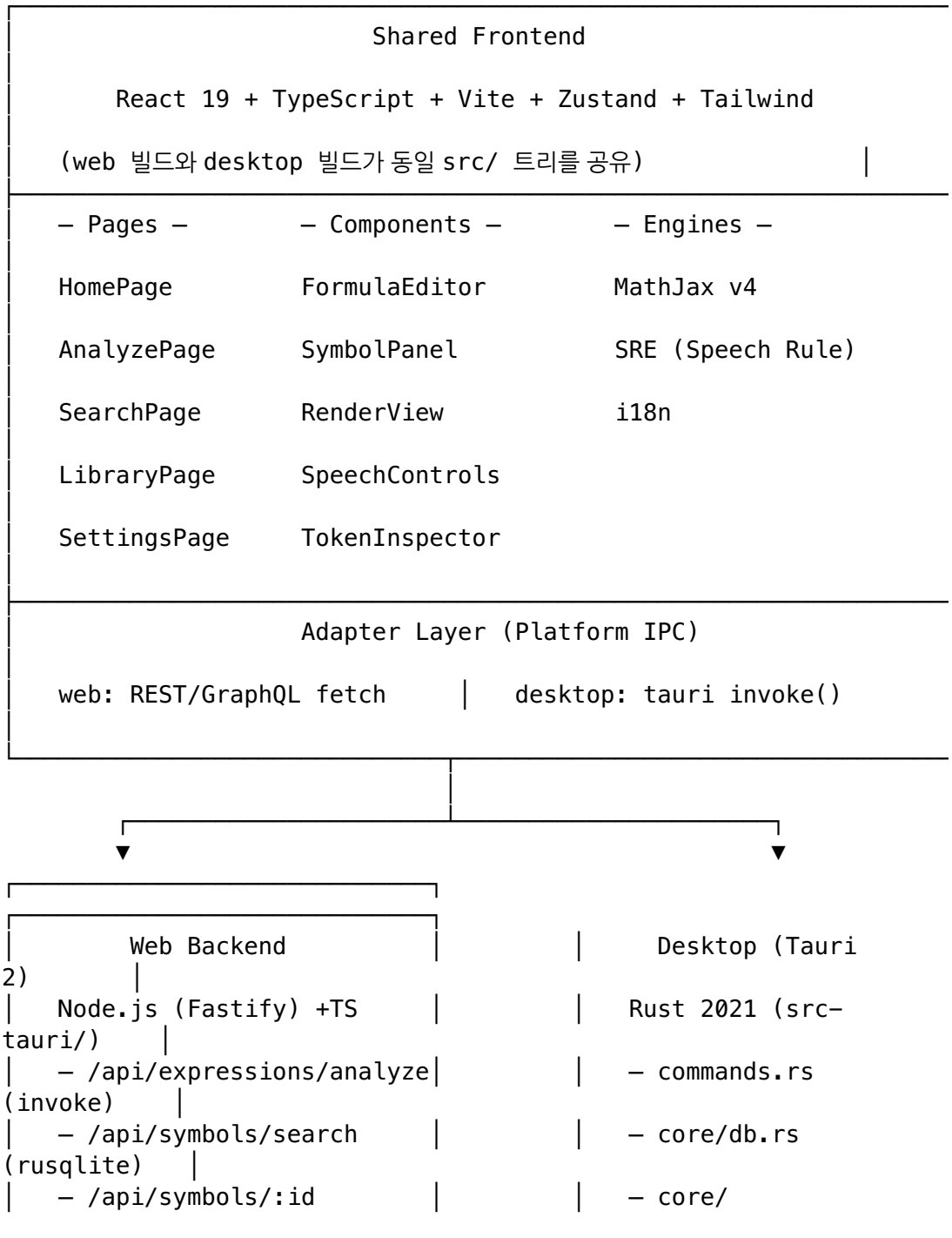
5. 비기능 요구사항(NFR)

카테고리	요구사항
성능	단일 수식 입력 → 렌더링 + 토큰화 < **300ms** (P95). 전체 읽기 생성 < **500ms**. 데스크탑 cold start < **2s**.
정확도	시드 사전 500개 기호 중 한국어 명칭/뜻 정확도 ≥ **95%** (전문가 1차 검수).
접근성	WCAG 2.1 AA. Lighthouse Accessibility ≥ **90**.
호환성 (웹)	Chrome 110+, Firefox 110+, Safari 16+, Edge 110+.
호환성 (데스크탑)	macOS 11+ (Universal), Windows 10+ (x64), Ubuntu 20.04+ (x64/arm64).
번들 크기 (데스크탑)	설치 패키지 ≤ **80MB** (Whisper처럼 큰 모델 없음).
오프라인	데스크탑은 첫 실행 후 인터넷 0회 의존(OCR/외부 TTS만 옵션).
로깅/프라이버시	수식 입력 텔레메트리 기본 OFF. 옵트인 시에만 익명 통계 전송.
보안	Tauri capability 최소권한, CSP 설정. SQL 인젝션 방어(Prepared Statement).
국제화	UI: ko(기본) / en. 사전: 한/영 동시 제공. 확장 슬롯: ja, zh.

카테고리	요구사항
라이선스	본체 MIT 또는 Apache-2.0. 의존성: MathJax(Apache-2.0), SRE(Apache-2.0), pix2tex(MIT), Tauri(MIT/Apache), unicode-math 시드(LPPL).

6. 시스템 아키텍처

6.1 전체 구조도 (상위)



```
parser.rs           │
│   — /api/ocr (옵션)          │          │   — core/speech.rs (SRE
FFI)│
│                              │          │   — core/
dictionary.rs       │
│   PostgreSQL 15+ + pg_trgm   │          │   — SQLite (FTS5,
bundled)  │
│   Redis (캐시)               │          │   — OS TTS
bridge            │
│   (옵션) Python OCR worker   │          │   — (옵션) Python OCR
sidecar│
└──────────────────────────────┘
└──────────────────────────────┘
```

6.2 두 버전의 코드 공유 전략

- `src/`(React) 100% 공유. 환경 분기는 단 한 곳:

```ts
// src/ipc/client.ts (As-It-Sounds 패턴 차용)
export const ipc = import.meta.env.VITE_PLATFORM === "desktop"
  ? tauriClient
  : webHttpClient;
```

- 빌드 산출물:
  - `pnpm build:web` → `dist-web/` → Vercel / 정적 호스팅 배포
  - `pnpm tauri:build` → 플랫폼별 인스톨러 (As-It-Sounds의 빌드 매트릭스 그대로)

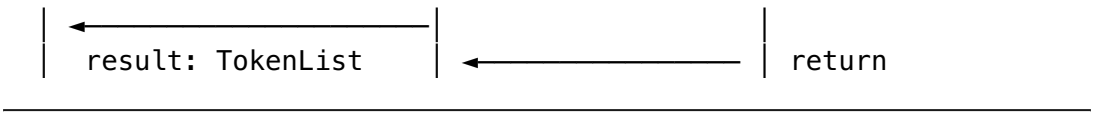
6.3 데이터 플로우

수식 분석 요청

```
[입력] → [정규화] → [MathJax 렌더] → [토큰 추출] → [SRE 읽기 생성]
                                                    ↓
                                        [DB 조회: symbols + domain_map]
                                                    ↓
                                          [응답: {render, speech, tokens[]}]
```

데스크탑 IPC 시퀀스 (As-It-Sounds 차용)

```
React                Tauri Bridge          Rust Core
  │                        │                    │
  │ invoke("analyze",.)    │                    │
  │ ──────────────────────►│ ──────────────────►│ parse + tokenize
  │                        │                    │ speech (SRE
bridge)
  │                        │                    │ dict_lookup
(SQLite)
  │  emit("token://hl")    │ ◄──────────────────│ stream highlight
evt
```



---

7. 데스크탑 앱 구조 — As-It-Sounds 준용

본 섹션은 ~/Developer/NGL(Go,Rust,...)/As-It-Sounds의 디렉토리·기술 스택·빌드 매트릭스를 분석한 결과를 직접 적용한다.

7.1 디렉토리 미러링

```
NSF-Reading-Helper/
├── package.json                # As-It-Sounds 동일 패턴
├── pnpm-lock.yaml
├── vite.config.ts
├── tsconfig.json
├── tailwind.config.js
├── postcss.config.js
├── index.html
├── public/
├── src/                        # React 19 + TS
│   ├── App.tsx                 # 라우팅 + bootstrap (As-It-Sounds
App.tsx 패턴)
│   ├── main.tsx
│   ├── index.css               # Tailwind
│   ├── types.ts                # 공유 타입
│   ├── pages/
│   │   ├── HomePage.tsx
│   │   ├── AnalyzePage.tsx     # ↔ ProjectPage 대응
│   │   ├── SearchPage.tsx
│   │   ├── LibraryPage.tsx
│   │   └── SettingsPage.tsx
│   ├── components/
│   │   ├── Layout.tsx
│   │   ├── Sidebar.tsx
│   │   ├── FormulaEditor.tsx   # 입력 (LaTeX/MathML/AsciiMath)
│   │   ├── RenderView.tsx      # MathJax 호스트
│   │   ├── SpeechControls.tsx  # 재생/속도/언어
│   │   ├── SymbolPanel.tsx     # 우측 설명 패널
│   │   ├── TokenInspector.tsx
│   │   ├── SearchBar.tsx
│   │   ├── ErrorBoundary.tsx
│   │   └── icons.tsx
│   ├── hooks/
│   │   ├── useMathJax.ts
│   │   ├── useSRE.ts
│   │   └── useTokenSelection.ts
│   ├── ipc/
│   │   └── client.ts           # ★ As-It-Sounds 동일 패턴 ★
│   ├── store/
│   │   ├── useAppStore.ts      # zustand, As-It-Sounds 패턴
```

```
│   │   └── usePipelineStore.ts  # 분석 진행 상태
│   ├── lib/
│   │   ├── mathjax/             # MathJax 부트스트랩
│   │   ├── sre/                 # SRE 래퍼
│   │   └── tokenizer.ts
│   └── mock/                    # 개발/스토리북용
├── src-tauri/                   # Rust
│   ├── Cargo.toml
│   ├── tauri.conf.json          # As-It-Sounds 키 패턴 그대로
│   ├── build.rs
│   ├── capabilities/
│   │   └── default.json
│   ├── entitlements.plist
│   ├── icons/
│   └── src/
│       ├── main.rs
│       ├── lib.rs               # tauri::Builder, AppState,
invoke_handler
│       ├── commands.rs          # #[tauri::command] 모음
│       └── core/
│           ├── mod.rs
│           ├── db.rs            # rusqlite bundled (As-It-Sounds
동일)
│           ├── error.rs         # AppResult, thiserror
│           ├── models.rs        # Project ↔ Workspace 대응
│           ├── settings.rs
│           ├── parser.rs        # LaTeX/MathML 파서 브릿지
│           ├── speech.rs        # SRE FFI 또는 sidecar JS 호출
│           ├── dictionary.rs    # 기호 사전 CRUD
│           ├── search.rs        # FTS5 + 트라이그램
│           ├── tts.rs           # OS TTS 브릿지
│           └── pack.rs          # .nsfpack import/export
├── Dockerfile.linux-build       # As-It-Sounds 빌드 패턴 그대로
├── .github/workflows/
│   └── release.yml              # 태그 푸시 → 멀티플랫폼 빌드
└── README.md
```

7.2 package.json 스크립트 (As-It-Sounds 준용)

```
{
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "build:web": "VITE_PLATFORM=web tsc && vite build --outDir
        dist-web",
    "preview": "vite preview",
    "tauri": "tauri",
    "tauri:dev": "tauri dev",
    "tauri:build": "tauri build",
    "tauri:build:mac": "tauri build --target universal-apple-
        darwin",
    "tauri:build:mac:arm": "tauri build --target aarch64-apple-
        darwin",
```

```
    "tauri:build:mac:intel": "tauri build --target x86_64-apple-
        darwin",
    "tauri:build:win": "tauri build --target x86_64-pc-windows-
        msvc",
    "tauri:build:linux": "tauri build --target x86_64-unknown-
        linux-gnu",
    "tauri:build:linux:arm": "tauri build --target aarch64-
        unknown-linux-gnu",
    "typecheck": "tsc --noEmit"
  }
}
```

7.3 Tauri 설정 (As-It-Sounds 동일 골격)

```
{
  "$schema": "https://schema.tauri.app/config/2",
  "productName": "NSF Reading Helper",
  "version": "0.1.0",
  "identifier": "com.nsfreading.app",
  "build": {
    "beforeDevCommand": "pnpm dev",
    "devUrl": "http://localhost:1420",
    "beforeBuildCommand": "pnpm build",
    "frontendDist": "../dist"
  },
  "app": {
    "windows": [{
      "title": "NSF Reading Helper",
      "width": 1280,
      "height": 820,
      "minWidth": 960,
      "minHeight": 640,
      "resizable": true,
      "center": true
    }],
    "security": { "csp": null, "assetProtocol": { "enable": true,
        "scope": ["**"] } }
  },
  "bundle": {
    "active": true,
    "targets": "all",
    "category": "Education",
    "shortDescription": "Reads natural-science formulas with
        Korean explanations",
    "longDescription": "Renders LaTeX/MathML/AsciiMath, speaks
        formulas in Korean using SRE, and explains every symbol
        per domain (math, physics, chemistry, biology, earth
        science).",
    "icon": [
      "icons/32x32.png",
      "icons/128x128.png",
      "icons/icon.icns",
      "icons/icon.ico"
```

```
    ]
  }
}
```

7.4 Cargo.toml (As-It-Sounds 차용 + 본 프로젝트 의존성)

```toml
[package]
name = "nsf-reading-helper"
version = "0.1.0"
edition = "2021"

[lib]
name = "nsf_reading_helper_lib"
crate-type = ["staticlib", "cdylib", "rlib"]

[build-dependencies]
tauri-build = { version = "2", features = [] }

[dependencies]
tauri = { version = "2", features = ["protocol-asset"] }
tauri-plugin-opener = "2"
tauri-plugin-dialog = "2"
tauri-plugin-fs = "2"

serde = { version = "1", features = ["derive"] }
serde_json = "1"

tokio = { version = "1", features = ["full"] }
reqwest = { version = "0.12", default-features = false, features
        = ["json", "rustls-tls"] }

rusqlite = { version = "0.32", features = ["bundled"] }

anyhow = "1"
thiserror = "2"
dirs = "5"
uuid = { version = "1", features = ["v4"] }
once_cell = "1"
parking_lot = "0.12"
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-
        filter"] }

# 본 프로젝트 추가
unicode-normalization = "0.1"
zip = "0.6"        # .nsfpack
regex = "1"

[profile.release]
codegen-units = 1
lto = true
```

```
opt-level = 3
strip = true
```

7.5 IPC 계약 (As-It-Sounds `ipc/client.ts` 패턴 그대로 + 본 프로젝트용 명령)

```ts
// src/ipc/client.ts
export const ipc = {
  // 분석
  analyze: (input: AnalyzeRequest) =>
        invoke<AnalyzeResult>("analyze", { input }),

  // 사전
  symbolSearch: (q: string, opts?: SearchOptions) =>
    invoke<SymbolHit[]>("symbol_search", { q, opts: opts ??
        null }),
  symbolGet: (id: string) => invoke<SymbolDetail>("symbol_get",
        { id }),
  symbolExamples: (id: string) =>
        invoke<SymbolExample[]>("symbol_examples", { id }),

  // 라이브러리
  historyList: () => invoke<HistoryItem[]>("history_list"),
  favoritesAdd: (symbolId: string) =>
        invoke<void>("favorites_add", { symbolId }),
  favoritesRemove: (symbolId: string) =>
        invoke<void>("favorites_remove", { symbolId }),
  notesUpsert: (n: Note) => invoke<Note>("notes_upsert", { note:
        n }),

  // 설정
  settingsGet: () => invoke<Settings>("settings_get"),
  settingsSet: (s: Settings) => invoke<Settings>("settings_set",
        { settings: s }),

  // 사전 팩
  packImport: (path: string) =>
        invoke<PackImportResult>("pack_import", { path }),
  packExport: (path: string) => invoke<void>("pack_export",
        { path }),

  // TTS
  ttsSpeak: (text: string, opts: TtsOptions) =>
    invoke<void>("tts_speak", { text, opts }),
  ttsStop: () => invoke<void>("tts_stop"),

  // OCR (sidecar 옵션)
  ocrImage: (filePath: string) => invoke<OcrResult>("ocr_image",
        { filePath }),
};

// 이벤트 (As-It-Sounds의 pipeline://event 패턴)
export function onAnalyzeEvent(handler: (e: AnalyzeEvent) =>
        void) {
```

```
  return listen<AnalyzeEvent>("analyze://event", (ev) =>
        handler(ev.payload));
}
```

7.6 데이터 저장 위치 (As-It-Sounds 패턴)

OS	경로
macOS	`~/Library/Application Support/com.nsfreading.app/`
Windows	`%APPDATA%\com.nsfreading.app\`
Linux	`~/.local/share/com.nsfreading.app/`

내부: - `nsf-reading.sqlite` — 사용자 데이터(히스토리/즐겨찾기/노트) + 임베드 사전 - `packs/` — 사용자 추가 사전 팩 - `cache/` — MathJax/SRE 캐시

7.7 빌드 매트릭스 (As-It-Sounds 동일 정책)

호스트	빌드 가능 타깃	명령
macOS (Apple Silicon)	macOS Universal / ARM / Intel	`tauri:build:mac` 등
Windows x64	Windows x64 (MSI/ NSIS)	`tauri:build:win`
Linux x64	deb/rpm/AppImage	`tauri:build:linux`
Linux arm64	deb/rpm/AppImage	`tauri:build:linux:arm`

크로스컴파일 한계는 As-It-Sounds와 동일(`ring`, `rusqlite` 네이티브 의존). **GitHub Actions로 멀티 OS 빌드** 권장.

7.8 핵심 차이점 (As-It-Sounds 대비)

항목	As-It-Sounds	NSF-Reading-Helper
도메인	오디오 음차	자연과학 수식
ASR	whisper-rs (네이티브)	**불필요**
미디어 처리	ffmpeg sidecar	**불필요** (선택적 OCR sidecar만)
핵심 엔진	Whisper + translit 모듈	**MathJax + SRE + 자체 사전 DB**
외부 호출	번역 API (MyMemory/ LibreTranslate)	**OCR 옵션** (pix2tex sidecar) 만
모델 다운로드	Whisper 모델 ~수GB	**없음** (사전 팩 ~수 MB)
번들 크기	큼 (whisper.cpp 포함)	**작음** (≤80MB)

8. 웹 버전 구조

8.1 기술 스택

- **Frontend**: Next.js 14 (App Router) + React 19 + TypeScript + Tailwind + Zustand.
- **Backend**: Node.js 20 + Fastify (또는 Next.js Route Handlers).
- **DB**: PostgreSQL 15+ (`pg_trgm`, `pg_jieba` 미사용, 한국어는 자체 토크나이저).
- **캐시**: Redis 7.
- **OCR 워커**: Python 3.11 + FastAPI + pix2tex (옵션, 컨테이너 분리).
- **배포**: Vercel (Frontend) + Fly.io/Render (API) + Neon/Supabase (Postgres).

8.2 API 명세 (REST 초안)

Method	Endpoint	설명
POST	`/api/expressions/analyze`	입력 수식 분석 (렌더 HTML + 읽기 + 토큰)
GET	`/api/symbols/search?q=&domain=&type=`	기호 검색
GET	`/api/symbols/:id`	기호 상세
GET	`/api/symbols/:id/examples`	분야별 예문
GET	`/api/domains`	도메인 목록
POST	`/api/ocr/formula`	이미지 → LaTeX (옵션)
POST	`/api/me/favorites`	즐겨찾기 추가 (로그인 필요)
GET	`/api/me/history`	분석 히스토리

POST `/api/expressions/analyze` 응답 예시

```
{
  "expressionId": 1024,
  "render": {
    "mathml": "<math>…</math>",
    "html": "<mjx-container …>…</mjx-container>"
  },
  "speech": {
    "ko": "0부터 1까지 x 제곱을 x에 대하여 적분",
    "en": "Integral from 0 to 1 of x squared d x"
  },
  "tokens": [
    {
      "index": 0,
      "tokenText": "\\int",
      "role": "operator",
      "symbolId": 101,
      "nameKo": "적분 기호",
```

```
      "readingKo": "인테그럴",
      "meaningKo": "연속적인 변화량의 누적",
      "domains": [
        { "code": "math-calculus", "meaning": "리만/르베그 적분" },
        { "code": "physics-general", "meaning": "전하·일·확률 등 누적
        량" }
      ]
    }
  ]
}
```

8.3 인증/계정

- V1: **회원가입 없이도 기본 기능 사용 가능** (분석/검색/읽기).
- 즐겨찾기·히스토리 동기화는 옵션(이메일/매직링크 또는 OAuth).
- 데스크탑은 기본 로컬-퍼스트, 옵션 클라우드 동기화는 V2+.

---

9. 데이터 모델 (DB 스키마)

> 참조 문서의 7-테이블 스키마를 기반으로 **본 프로젝트용 확장**을 적용.
> PostgreSQL(웹)과 SQLite(데스크탑) **양쪽이 동일 스키마**
> 를 공유하며, DDL 차이는 마이그레이션 도구로 흡수.

9.1 테이블 ERD (개념)

```
domains ──< symbol_domain_map >── symbols ──< symbol_aliases
                                          │
                                          ├──< symbol_examples
                                          │
expressions ──< expression_symbols >──┘

users (선택, 웹만) ──< favorites, history, notes
```

9.2 핵심 DDL (PostgreSQL 기준 — SQLite는 타입만 치환)

domains

```sql
create table domains (
  id           bigserial primary key,
  code         varchar(50) unique not null,
  name_ko      varchar(100) not null,
  name_en      varchar(100),
  description  text,
  created_at   timestamptz default now()
);
```

symbols

```
create table symbols (
  id                  bigserial primary key,
  unicode_char        varchar(16),
  latex_command       varchar(128),
  mathml_tag          varchar(128),
  canonical_name_ko   varchar(200) not null,
  canonical_name_en   varchar(200),
  reading_ko          varchar(200),
  reading_en          varchar(200),
  short_meaning_ko    text not null,
  short_meaning_en    text,
  symbol_type         varchar(50) not null,
  difficulty_level    smallint default 1,
  is_operator         boolean default false,
  is_relation         boolean default false,
  is_delimiter        boolean default false,
  is_greek            boolean default false,
  is_constant         boolean default false,
  is_function         boolean default false,
  notes               text,
  created_at          timestamptz default now(),
  updated_at          timestamptz default now()
);
```

symbol_aliases / symbol_domain_map / expressions / expression_symbols / symbol_examples

> 참조 문서의 동일 DDL을 그대로 채택.

(본 프로젝트 추가) favorites

```
create table favorites (
  id          bigserial primary key,
  user_id     bigint references users(id) on delete cascade,
  symbol_id   bigint not null references symbols(id) on delete
        cascade,
  created_at  timestamptz default now(),
  unique(user_id, symbol_id)
);
```

(본 프로젝트 추가) notes

```
create table notes (
  id            bigserial primary key,
  user_id       bigint references users(id) on delete cascade,
  expression_id bigint references expressions(id) on delete set
        null,
  symbol_id     bigint references symbols(id) on delete set null,
  body_md       text not null,
  tags          text[] default '{}',
```

```
  created_at    timestamptz default now(),
  updated_at    timestamptz default now()
);
```

(데스크탑 전용 SQLite) `app_settings`

```
create table app_settings (
  key   text primary key,
  value text not null
);
```

9.3 인덱스

```
create index idx_symbols_unicode_char on symbols(unicode_char);
create index idx_symbols_latex_command on symbols(latex_command);
create index idx_symbols_symbol_type on symbols(symbol_type);
create index idx_aliases_alias_text on symbol_aliases(alias_text);
create index idx_expr_symbols_expression on
        expression_symbols(expression_id);

-- 퍼지 검색 (Postgres)
create extension if not exists pg_trgm;
create index idx_aliases_trgm on symbol_aliases using gin
        (alias_text gin_trgm_ops);
create index idx_symbols_name_trgm on symbols using gin
        (canonical_name_ko gin_trgm_ops);

-- SQLite FTS5 (Desktop)
create virtual table symbols_fts using fts5(
  canonical_name_ko, canonical_name_en, latex_command,
        unicode_char, aliases,
  content=''
);
```

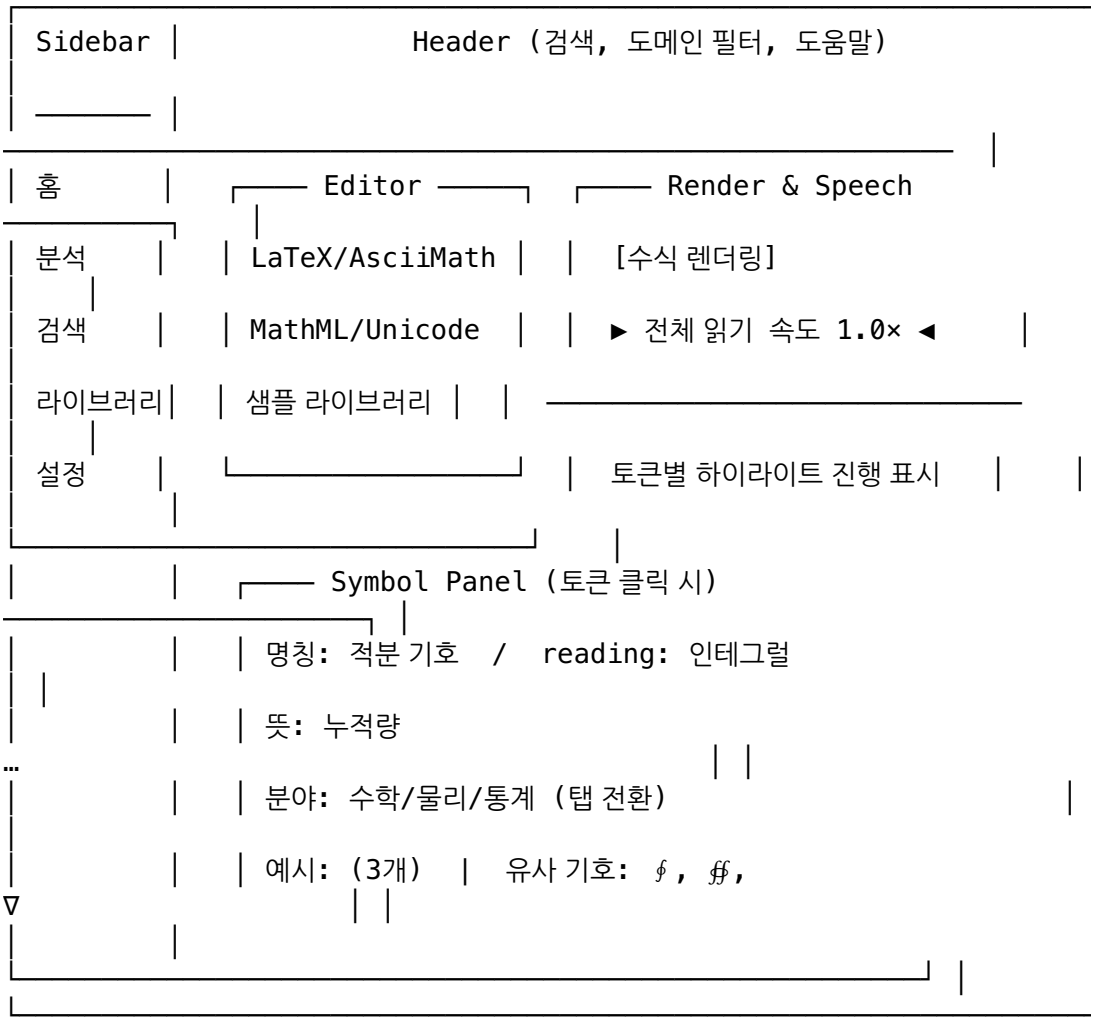
9.4 초기 시드 (V1 최소)

- 도메인 7개: `math-algebra`, `math-calculus`, `math-set-logic`, `physics-general`, `chemistry`, `earth-science`, `biology`, `statistics`, `logic`.
- 기호 500개 이상:
  - 그리스 문자 대/소 48개
  - 기본 연산자 (+, −, ×, ÷, ±, …) 약 20개
  - 비교/관계 (=, ≠, <, ≤, ≈, ≡, …) 약 25개
  - 집합/논리 (∈, ∉, ⊂, ⊆, ∪, ∩, ∀, ∃, …) 약 30개
  - 미적분 (∫, ∮, ∑, ∏, ∂, ∇, lim, …) 약 25개
  - 물리 상수/단위 (c, h, ħ, k_B, ε0, μ0, kg, ℳℓ, …) 약 60개
  - 화학 표기 (→, ⇌, 상태 표기 (aq), (s), mhchem `\ce{}`) 약 40개
  - 생물·통계 (μ, σ, χ^2, p, r^2) 약 30개
  - 지구과학 (ω, °, ′, ″, …) 약 30개
  - 함수 (sin, cos, log, ln, exp) 약 30개
  - 첨자/위첨자/벡터/행렬 표기 약 30개
  - 추가 화살표/관계 약 30개

---

10. UX & UI 가이드

10.1 핵심 레이아웃 (1280×820)



10.2 인터랙션 규칙

- 토큰 클릭 → SymbolPanel 갱신 + 토큰 강조.
- 키보드: Tab으로 토큰 순회, Enter로 읽기, Space로 일시정지.
- 검색창은 헤더 고정, `Cmd/Ctrl+K`로 포커스.
- 다크 모드 기본 ON (자연과학 PDF 대비 눈 피로 최소화).

10.3 접근성

- 모든 수식 컨테이너에 `role="math"`, `aria-label`은 SRE 출력 한국어 텍스트.
- 토큰은 `tabindex="0"` + 포커스 링.
- TTS 재생 중 진행도를 `aria-live="polite"`로 알림.

---

11. 보안 & 프라이버시

- **Tauri Capabilities**: `dialog`, `fs:default(scope: app-data-only)`, `opener` 만 허용.
- **CSP**: 외부 스크립트 차단(MathJax/SRE는 번들).
- **OCR sidecar** 호출은 사용자 명시 동의 후에만 활성화.
- **텔레메트리**: opt-in. 익명 해시된 입력 길이/도메인 분포만 전송. 원문 미전송.
- **로그**: PII 비저장.

---

12. 분석/메트릭

지표	목표
DAU/WAU	(출시 1개월) 100/300
평균 세션당 분석 수식 수	5+
사전 검색 → 클릭 전환율	≥ 60%
토큰 클릭 비율 (수식 내 토큰의 30% 이상 탐색)	≥ 40%
접근성 Lighthouse	≥ 90
데스크탑 crash-free 세션	≥ 99%

---

13. 마일스톤 & 로드맵

M0 — 셋업 (2주)

- Repo 부트스트랩, As-It-Sounds 디렉토리 미러링, CI 파이프라인.
- Tauri 2 `init`, MathJax v4 embed, SRE 동작 확인.
- DB DDL 적용 (Postgres + SQLite 양쪽), 마이그레이션 도구.

M1 — 코어 분석 파이프라인 (3주)

- LaTeX/MathML/AsciiMath 입력 → 정규화 → 렌더링 → 읽기 생성.
- 토큰 추출 + DB 매핑.
- 기본 SymbolPanel.

M2 — 시드 사전 500개 (4주, 병행)

- 콘텐츠 큐레이션(전문가 1인 + 검수 1인).
- domains/symbols/aliases/examples 시딩 스크립트.

M3 — 검색/라이브러리/접근성 (3주)

- 통합 검색, 퍼지 매칭, 분야 필터.
- 즐겨찾기/히스토리/노트.

• A11y 패스(WCAG 2.1 AA).

M4 — 데스크탑 패키징 (2주)

• macOS Universal / Windows / Linux 빌드.
• 자동 업데이트, 코드 서명.
• `.nsfpack` import/export.

M5 — 웹 배포 (2주)

• Next.js → Vercel, API → Fly.io, DB → Neon.
• 분석/모니터링(Sentry, Plausible).

M6 — 베타 → V1.0 (2주)

• 50명 베타 피드백 반영.
• 정식 릴리스 v1.0.0.

V1.5 (8주)

• OCR(pix2tex sidecar), 손그림 검색, 화학식(mhchem) 확장.

총 V1 약 **16주(4개월)** 예상.

---

14. 리스크 & 대응

리스크	영향	대응
SRE 한국어 규칙 부재로 읽기 품질 저하	높음	자체 `ko-clearspeak.json` 규칙팩 작성, 사용자 피드백 학습
분야별 의미 모호성 (μ, σ 등)	중간	`symbol_domain_map` 다대다 + 사용자 선택 컨텍스트 저장
MathJax v4 SRE 통합 안정성	중간	v3 폴백 옵션, e2e 회귀 테스트
Tauri 크로스컴파일 (As-It-Sounds 동일 한계)	중간	GitHub Actions 멀티 OS 매트릭스
사전 콘텐츠 라이선스	높음	unicode-math는 시드만, 설명/예문은 자체 큐레이션, 라이선스 명시
시각장애 사용자 검증 부재	중간	베타 단계 협력자 확보, 스크린리더 실측

---

15. 미해결 질문 (Open Questions)

1. 웹/데스크탑 사용자 데이터 동기화: V1에 넣을지, V2로 미룰지?
2. 한국어 읽기 규칙의 권위 출처(교과부/학회 표기 기준) 확정 필요.
3. 화학 분야 표기는 mhchem만 다룰지, SMILES까지 포함할지?
4. 모바일 PWA 우선(V1.5)인지, V2로 미룰지?
5. 사전 콘텐츠 기여를 외부 오픈소스화할지(MIT 콘텐츠)?

---

16. 합의(Approval)

역할	이름	승인
Product Owner	TBD	☐
Tech Lead	TBD	☐
Design Lead	TBD	☐
Accessibility Reviewer	TBD	☐

부록 A — 참고 자료

- 본 저장소 `.doc/Perplexity-수학계열, 물리,화학,지구과학,생물 등의 자연과학 포함하여 다양한 수식.md` (오픈소스 비교/스키마 시드)
- `~/Developer/NGL(Go,Rust,...)/As-It-Sounds` — 데스크탑 앱 구조 레퍼런스
  - `package.json`, `src-tauri/tauri.conf.json`, `src-tauri/Cargo.toml`
  - `src/App.tsx`, `src/ipc/client.ts`, `src/store/useAppStore.ts`
  - `src-tauri/src/lib.rs`, `src-tauri/src/commands.rs`, `src-tauri/src/core/*`
- 외부:
  - MathJax: https://www.mathjax.org
  - Speech Rule Engine: https://speechruleengine.org
  - pix2tex: https://github.com/lukas-blecher/LaTeX-OCR
  - Tauri 2 docs: https://tauri.app

부록 B — 용어

용어	정의
MathJax	LaTeX/MathML/AsciiMath를 브라우저에서 렌더링하는 JavaScript 라이브러리
SRE	Speech Rule Engine. 수식을 사람이 읽을 자연어 문자열로 변환
Token	수식을 구성하는 최소 의미 단위(연산자, 식별자, 관계기호 등)
Clearspeak/Mathspeak	

용어	정의
	SRE가 제공하는 두 가지 읽기 규칙. Clearspeak가 더 자연스러움
.nsfpack	본 프로젝트의 사전 패키지 포맷(zip + manifest.json)
Domain	수학/물리/화학/생물 등 분야 분류